



Debezium PostgreSQL Connector: Real-Time CDC with Streaming SQL



RisingWave Labs · Apr 3, 2026 · 5 min read

The Debezium PostgreSQL connector reads from PostgreSQL's logical replication stream and publishes every row-level change to Kafka, giving downstream consumers a real-time feed of your database mutations with full before/after state. Combined with RisingWave's streaming SQL, you get live analytics and data products on top of your Postgres workload.

Why Use CDC with PostgreSQL?

PostgreSQL is a workhorse OLTP database, but its strength — handling high-concurrency transactional writes — creates a tension with analytics. Running heavy analytical queries on your production Postgres instance competes with application traffic.

CDC solves this by streaming changes out of Postgres asynchronously via its Write-Ahead Log (WAL). The WAL is the mechanism Postgres uses for crash recovery and replication; Debezium plugs into it using the logical replication protocol, which is designed for exactly this use case and has negligible impact on the primary.

Key use cases for Postgres CDC:

- Keeping a data warehouse or search index in sync with the primary
- Triggering real-time notifications when specific rows change
- Building audit logs with full before/after state
- Feeding streaming SQL engines like RisingWave for live aggregations

How the Debezium PostgreSQL Connector Works

The connector uses PostgreSQL's logical decoding feature. A **replication slot** is created on the primary, which tells Postgres to retain WAL segments until the slot's consumer (Debezium) has processed them.

Debezium supports two logical decoding output plugins:

- `pgoutput` : Built into PostgreSQL 10+. No additional extensions required.
- `decoderbufs` : A third-party plugin that produces a Protobuf-encoded stream; requires installation.

For most deployments, `pgoutput` is recommended because it ships with Postgres itself.

A **publication** defines which tables Debezium will monitor. You can monitor all tables or a specific subset.

Step-by-Step Tutorial

Step 1: Prepare PostgreSQL

Enable logical replication in `postgresql.conf` :

```
wal_level = logical  
max_replication_slots = 4  
max_wal_senders = 4
```

Create a dedicated replication user and set up a publication:

```
-- Create replication user  
CREATE ROLE debezium REPLICATION LOGIN PASSWORD 'secret';  
GRANT SELECT ON ALL TABLES IN SCHEMA public TO debezium;  
  
-- Create publication for the tables you want to capture  
CREATE PUBLICATION dbz_publication FOR TABLE public.orders, public.customers;
```

Step 2: Deploy the Debezium Connector

Register the PostgreSQL connector with Kafka Connect:

```
{  
  "name": "pg-orders-connector",
```

```

"config": {
  "connector.class": "io.debezium.connector.postgresql.PostgresConnector",
  "database.hostname": "postgres.internal",
  "database.port": "5432",
  "database.user": "debezium",
  "database.password": "secret",
  "database.dbname": "production",
  "database.server.name": "pg1",
  "slot.name": "debezium_slot",
  "plugin.name": "pgoutput",
  "publication.name": "dbz_publication",
  "table.include.list": "public.orders,public.customers",
  "topic.prefix": "pg1",
  "heartbeat.interval.ms": "10000"
}
}

```

```

curl -X POST http://kafka-connect:8083/connectors \
  -H "Content-Type: application/json" \
  -d @pg-connector.json

```

Debezium will perform an initial snapshot (reading current rows as `op=r` events), then switch to streaming mode.

Step 3: Connect RisingWave to the CDC Topic

RisingWave is a PostgreSQL-compatible streaming database. Connect it to the Kafka topic produced by Debezium:

```

CREATE SOURCE orders_source
WITH (

```

```
connector = 'kafka',  
topic = 'pg1.public.orders',  
properties.bootstrap.server = 'kafka:9092',  
scan.startup.mode = 'earliest'  
) FORMAT DEBEZIUM ENCODE JSON;
```

RisingWave parses the Debezium envelope — `before`, `after`, `op`, `ts_ms` — and maintains the current state of the table automatically.

Step 4: Build Materialized Views

Create streaming aggregations that update as each CDC event arrives:

```
CREATE MATERIALIZED VIEW customer_order_stats AS  
SELECT  
  o.customer_id,  
  COUNT(*) AS total_orders,  
  SUM(o.amount) AS lifetime_value,  
  MAX(o.created_at) AS last_order_at  
FROM orders_source o  
GROUP BY o.customer_id;
```

Query it like any Postgres view — results always reflect the latest CDC events:

```
SELECT * FROM customer_order_stats  
WHERE lifetime_value > 1000  
ORDER BY lifetime_value DESC  
LIMIT 10;
```

Step 5: Sink to Downstream Systems

Push the aggregated results to another Kafka topic for consumption by dashboards or other services:

```
CREATE SINK customer_stats_sink
FROM customer_order_stats
WITH (
  connector = 'kafka',
  topic = 'customer-order-stats',
  properties.bootstrap.server = 'kafka:9092'
) FORMAT DEBEZIUM ENCODE JSON;
```

Comparison Table

Aspect	Debezium + Kafka + RisingWave	RisingWave Native Postgres CDC
Setup complexity	Medium (Kafka Connect + connector config)	Low (one CREATE SOURCE statement)
Kafka dependency	Required	Not required
Fan-out (multiple consumers)	Easy — any Kafka consumer reads the same topic	Requires separate connections per consumer
Replication slot management	Managed by Debezium	Managed by RisingWave
Initial snapshot	Built-in	Built-in
Best for	Multi-consumer architectures	Single-pipeline simplicity

FAQ

What is a replication slot and why does it matter? A replication slot is a Postgres mechanism that ensures WAL segments are retained until all registered consumers have processed them. Debezium creates and manages one slot per connector. If Debezium stops consuming, the slot causes WAL to accumulate. Monitor slot lag and set `max_slot_wal_keep_size` to protect disk space.

Can I monitor only specific columns? Debezium captures full row changes by default. You can use `column.exclude.list` to drop sensitive columns (like passwords) from the event payload before they reach Kafka.

Does the connector handle DDL changes? Debezium tracks schema changes via PostgreSQL's schema history topic. When a column is added or renamed, Debezium updates its internal schema and continues processing without interruption.

Key Takeaways

- Debezium reads Postgres WAL via logical replication slots using `pgoutput` or `decoderbufs`
- Configure with `database.hostname`, `slot.name`, `plugin.name`, and `table.include.list`
- RisingWave ingests Debezium events using `FORMAT DEBEZIUM ENCODE JSON` on Kafka sources
- Materialized views in RisingWave update incrementally on each CDC event
- Use Debezium + Kafka when you need fan-out; use RisingWave's native `postgres-cdc` connector for simpler setups

[Get started](#) | [Slack](#)

Best-in-Class Event Streaming for Agents, Apps, and Analytics

[START FREE](#)[JOIN OUR SLACK](#)

Sign up for our [newsletter](#) to stay updated.



95 3RD ST, 2ND FLOOR,
SAN FRANCISCO, CA 94103



PRODUCTS

[Overview](#)[RisingWave Ultra](#)[Open Lake](#)[Low-Latency
Serving](#)[Cloud](#)[Self-Managed](#)[Pricing](#)

KEY CAPABILITIES

[Real-Time Event Processing](#)[Incremental Materialized
Views](#)[Low-Latency Serving](#)[Apache Iceberg™
Maintenance](#)

INDUSTRIES

[Capital Markets](#)[Financial
Services](#)[Sports Betting](#)[E-Commerce](#)[Healthcare](#)[Manufacturing](#)[Logistics](#)[AD/Marketing
Tech](#)

USE CASES

[Monitoring and Alerting](#)[Real-Time Data
Enrichment](#)[IoT and Telemetry
Pipelines](#)[Streaming Lakehouse](#)

RESOURCES

[Documentation](#)[Guides](#)[Live Demos](#)[Connectors](#)[Glossary](#)[Blog](#)[Community](#)[Customer
Stories](#)

COMPARE

[vs Apache Kafka](#)[vs Apache Flink](#)[vs Spark
Streaming](#)[vs ksqldb](#)[vs ClickHouse](#)[vs Debezium](#)

COMPANY

[About Us](#)[Careers](#)[Contact Us](#)[Partners](#)[Privacy Policy](#)[Terms of](#)[Service](#)

